KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 1
변수, 타입, 연산

값에 이름을 붙이고,
계산하고, 비교하기

```
price = 4500

type(price)          ➡ int

price * 2            ➡ 9000

price * 2 >= 10000   ➡ False
```

오늘의 순서

0	들어가기	왜 변수가 필요한지, 큰 그림, 노트북 준비	3–5쪽
1	변수에 값 넣기	= 로 넣기, 값 보기, 다시 넣기, NameError	6–11쪽
2	변수 타입	int, float, str, bool, type(), 타입 바꾸기	12–16쪽
3	사칙연산	+ - * /, 몫과 나머지, 계산 순서, +=, 글자 연산	17–27쪽
4	비교와 논리연산	비교 여섯 가지, = 와 ==, and, or, not	28–35쪽
5	카페 주문으로 풀어 보기	합계와 잔돈, 나눠 내기, 할인 조건	36–38쪽
	정리	막힐 때, 한 장 요약	39–40쪽

왜 필요한가

숫자만 적으면 무슨 계산인지 잊어요

[1]
```python
print(4500 * 2 + 5000)
```
Python

```
14000
```

← 아메리카노 두 잔과 라떼 한 잔

[2]
```python
print(20000 - (4500 * 2 + 5000))
```
Python

```
6000
```

← 2만 원을 내면 잔돈은? 같은 식을 또 적어요

아메리카노가 4800원이 되면 4500을 찾아 두 줄 다 고쳐요

숫자에 이름을 붙여 두면 읽기 쉽고, 고칠 곳도 한 곳이에요. 그 이름이 변수예요.

큰 그림

값을 상자에 담고, 계산하고, 비교해요

1 변수

americano

4500

```
americano = 4500
```

2 타입

4500	int
"라떼"	str
True	bool

```
type(4500)
```

3 계산

total

9000

```
total = americano * 2
```

4 비교

```
9000 >= 10000
```

False

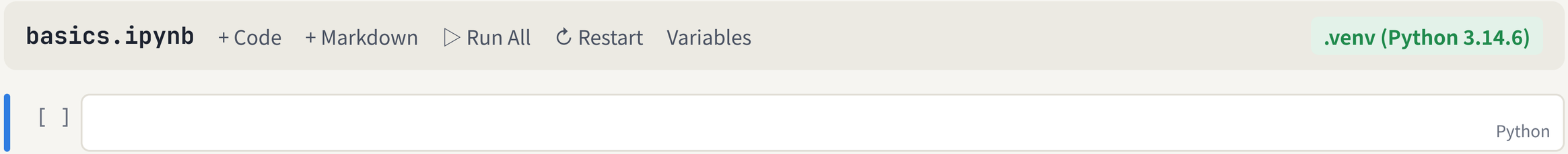
```
total >= 10000
```

---

5 카페 주문 네 가지를 섞어 합계와 잔돈, 나눠 내기, 할인 조건을 풀어요

들어가기

hello-uv에 새 노트북 basics.ipynb를 만들어요



1. VS Code로 hello-uv 폴더를 열어요
2. 탐색기에서 새 파일 basics.ipynb를 만들어요
3. 오른쪽 위 커널로 .venv를 골라요
4. 셀에 적고 Shift+Enter로 실행해요 — `Shift` + `Enter`

기억이 안 나면

커널 .venv 고르기 — 09 · 25–27쪽

변수 이름 규칙 — 10 · 24–26쪽

자료형과 type() — 10 · 27–28쪽

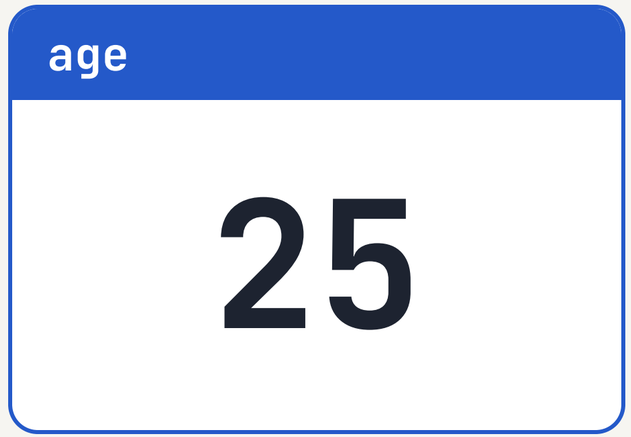
1. 변수에 값 넣기

= 는 오른쪽 값을 왼쪽 이름에 넣어요

```
[1]  age = 25                                    Python
```

넣기만 해서 출력은 없어요. 값은 커널이 기억해요





수학의 '같다'와 뜻이 달라요. 25를 age라는 상자에 넣어요.

1. 변수에 값 넣기

넣은 값은 두 가지 방법으로 봐요

셀 마지막 줄에 이름만

[2]
```python
age
```

25 ← 마지막 줄이 값이면 그대로 보여요

print()로 꺼내기

[3]
```python
print(age)
print("나이:", age)
```

```
25
나이: 25
```

```
print("나이:", age)
```

따옴표 안 글자는 그대로

값 25로 바뀌고, 쉼표 자리는 한 칸

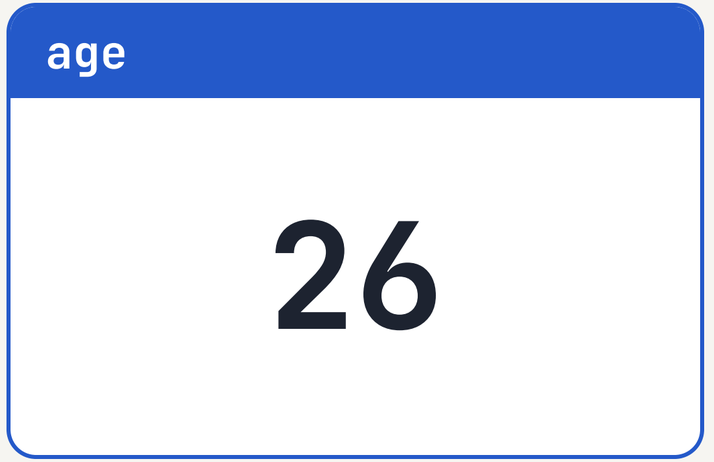
1. 변수에 값 넣기

다시 넣으면 새 값으로 바뀌어요

[4]
```
age = 26    ← 같은 이름에 새 값
age
```
Python

```
26
```

셀	넣은 순서	
[1]	age = 25	처음 넣은 값
[4]	age = 26	지금 남은 값



상자 하나에는 값이 하나만 들어가요. 마지막에 넣은 값만 남아요.

1. 변수에 값 넣기

age = age + 1은 오른쪽부터 계산해요

[5]
```
age = age + 1
age
```
Python

```
27
```

age
27

age = **age** + **1**

③ 27을 넣어요 ① 지금 값 26 ② 1을 더해 27

10에서 실행한 x = x + 1도 같은 순서예요.

1. 변수에 값 넣기

이름만 적으면 마지막 줄만 보여요

[6]
```
user_name = "kim"
age                    ← 실행돼도 안 보여요
user_name              ← 이 줄만 보여요
```
Python

```
'kim'
```

[7]
```
print(age)
print(user_name)       ← 둘 다 보여요
```
Python

```
27
kim
```

'kim' 이름만 적으면 따옴표까지

kim print는 글자만

여러 값을 보려면 줄마다 print()를 써요.

1. 변수에 값 넣기

이름을 틀리게 적으면 NameError가 나요

```
[8]  Age                                                    Python
```

```
NameError: name 'Age' is not defined
```

← 모르는 이름이에요

✕ ~~Age~~ → ✓ `age` 대문자와 소문자는 다른 이름이에요

이럴 때도 같은 오류가 나요

철자를 틀렸어요	agee
넣는 셀을 아직 실행하지 않았어요	넣는 셀부터 실행
Restart 뒤에 아래 셀만 실행했어요	▷ Run All

2. 변수 타입

값마다 타입이 있어요

[1]
```python
age = 25                    ← 정수 · 소수점 없는 숫자
height = 172.5              ← 실수 · 소수점 있는 숫자
user_name = "kim"           ← 문자열 · 따옴표로 감싼 글자
is_member = False           ← 불리언 · True 또는 False
print(type(age))
print(type(height))
print(type(user_name))
print(type(is_member))
```

Python

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

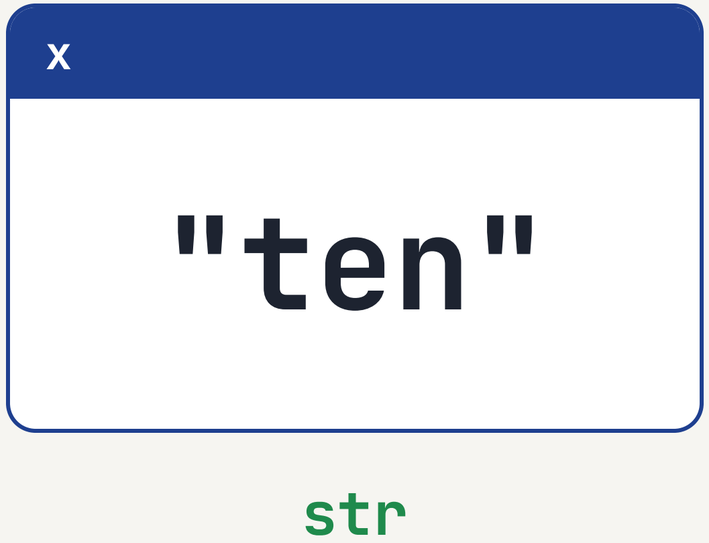
타입은 값의 종류예요. 10에서 본 자료형과 같은 말이에요.

2. 변수 타입

타입은 이름이 아니라 값을 따라가요

[2]
```python
x = 10
print(type(x))
x = "ten"
print(type(x))
```

```
<class 'int'>
<class 'str'>
```



이름 x는 그대로예요. 넣은 값이 바뀌니 타입도 바뀌었어요.

2. 변수 타입

따옴표 하나로 숫자가 글자가 돼요

[3]
```python
print(type(25))
print(type("25"))
print(25 + 1)
print("25" + "1")
```

```
<class 'int'>
<class 'str'>
26
251
```

25 + 1 → 26

int · 숫자끼리 더해요

"25" + "1" → 251

str · 따옴표가 있으면 글자끼리 이어 붙여요

겉모습이 숫자여도 따옴표 안에 있으면 글자예요.

2. 변수 타입

int(), float(), str()로 타입을 바꿔요

[4]
```python
print(int("3") + 3)
print(str(3) + "3")
print(float("3.5"))
print(int(3.9))
```

```
6
33
3.5
3
```

바꾸기	결과	뜻
`int("3")`	`3`	글자를 정수로
`str(3)`	`'3'`	숫자를 글자로
`float("3.5")`	`3.5`	글자를 실수로
`int(3.9)`	`3`	소수점 아래는 버려요

바꾸고 싶은 타입 이름을 함수처럼 쓰고 괄호 안에 값을 넣어요.

2. 변수 타입

숫자로 못 읽는 글자는 ValueError가 나요

[5]
```
int("삼")
```
Python

```
ValueError: invalid literal for int() with base 10: '삼'
```
← 한글 숫자는 못 읽어요

[6]
```
int("3.5")
```
Python

```
ValueError: invalid literal for int() with base 10: '3.5'
```
← 소수점 글자도 못 읽어요

× ~~int("3.5")~~ → ✓ `float("3.5")` → 3.5

ValueError는 타입은 맞는데 그 값으로는 바꿀 수 없다는 뜻이에요.

3. 사칙연산

더하기, 빼기, 곱하기, 나누기

[1]
```python
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 3)
print(6 / 2)
```

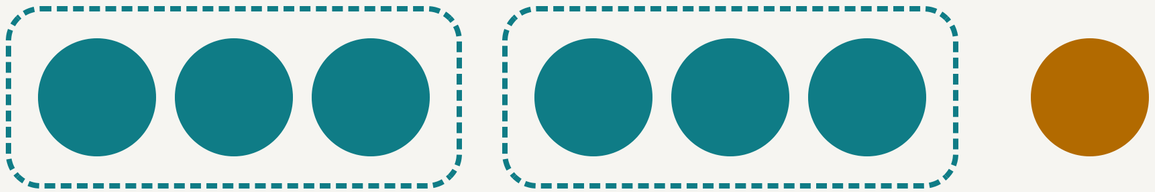
```
10
4
21
2.3333333333333335  ← 끝자리가 조금 어긋나요
3.0                 ← 나눠떨어져도 .0
```

기호	뜻	예	결과
+	더하기	7 + 3	10
-	빼기	7 - 3	4
*	곱하기	7 * 3	21
/	나누기 결과는 늘 실수	7 / 3	2.333…

3. 사칙연산

몫 //, 나머지 %, 거듭제곱 **

사탕 7개를 3개씩 나누면



몫 2 (묶음 수)　　나머지 1

```
[2] print(7 // 3)
    print(7 % 3)
    print(2 ** 3)
```

Python

```
2
1
8
```

//	몫	7 // 3 → 2	나누고 소수점 아래는 버려요
%	나머지	7 % 3 → 1	나누고 남은 수. 퍼센트가 아니에요
**	거듭제곱	2 ** 3 → 8	2 × 2 × 2

3. 사칙연산

곱하기를 더하기보다 먼저 계산해요

[3]
```python
2 + 3 * 4
```
Python

```
14
```

[4]
```python
(2 + 3) * 4
```
Python

```
20
```

2 + 3 * 4 곱하기 먼저

2 + 12 그다음 더하기

14

(2 + 3) * 4 괄호 먼저

5 * 4 그다음 곱하기

20

더하기를 먼저 하고 싶으면 그 부분을 괄호로 묶어요.

3. 사칙연산

계산 순위는 네 계단이에요

1 ( )

괄호

가장 먼저

2 **

거듭제곱

3 * / // %

곱하기, 나누기, 몫, 나머지

넷은 같은 순위. 몫과 나머지도 여기예요

4 + -

더하기, 빼기

가장 나중

위 계단부터 풀어요. 같은 계단끼리는 왼쪽부터예요.

3. 사칙연산

** 는 곱하기와 몫보다 먼저 계산해요

[5]
```
print(2 ** 3 // 3)
print(2 * 3 ** 2)
print(-2 ** 2)
print((-2) ** 2)
```
Python

```
2
18
-4
4
```

`2 ** 3` `//` `3` ** 먼저

`8 // 3` 그다음 몫

`2`

`2 *` `3 ** 2` ** 먼저

`2 * 9` 그다음 곱하기

`18`

`-2 ** 2` ** 가 빼기 기호보다 먼저

`-4`

`(-2)` `**` `2` 괄호 먼저

`4`

-2 ** 2는 -4예요. 음수를 거듭제곱하려면 (-2)처럼 괄호로 묶어요.

3. 사칙연산

같은 순위끼리는 왼쪽부터 풀어요

[6]
```python
print(10 - 4 + 2)
print(8 / 2 * 4)
print(7 // 2 * 2)
```

```
8
16.0
6
```

`10 - 4` + 2 → `6 + 2` → `8`

왼쪽 빼기 먼저

⚠ 10 - 6 = 4가 아니에요

`8 / 2` * 4 → `4.0 * 4` → `16.0`

왼쪽 나누기 먼저

⚠ 8 / 8 = 1이 아니에요

`7 // 2` * 2 → `3 * 2` → `6`

왼쪽 몫 먼저

⚠ // 와 * 도 같은 순위

뒤쪽을 먼저 하고 싶으면 그 부분을 괄호로 묶어요.

3. 사칙연산

섞여 있으면 한 계단씩 풀어요

```
[7] print(10 - 2 * 3 ** 2 // 4)                                    Python
```

```
6
```

`10 - 2 * 3 ** 2 // 4` 2단계 ** 부터

`10 - 2 * 9 // 4` 3단계, 왼쪽부터

`10 - 18 // 4` 3단계

`10 - 4` 4단계

`6`

직접 짤 땐 10 - (2 * 3 ** 2) // 4처럼 괄호로 순서를 보여 줘요.

계단 순서

1. ( ) 괄호가 없어 건너뛰어요
2. **
3. * / // %
4. + -

3. 사칙연산

괄호 하나로 답이 달라져요

괄호 없이	결과	괄호를 치면	결과
`10 + 2 * 3`	`16`	`(10 + 2) * 3`	`36`
`20 - 10 / 2`	`15.0`	`(20 - 10) / 2`	`5.0`
`12 / 3 * 2`	`8.0`	`12 / (3 * 2)`	`2.0`
`100 - 30 - 20`	`50`	`100 - (30 - 20)`	`90`
`2 ** 2 * 3`	`12`	`2 ** (2 * 3)`	`64`
`17 % 5 * 2`	`4`	`17 % (5 * 2)`	`7`
`3 + 10 // 3`	`6`	`(3 + 10) // 3`	`4`
`-3 ** 2`	`-9`	`(-3) ** 2`	`9`

%, // 는 곱하기와 같은 순위, ** 는 그보다 먼저예요.

3. 사칙연산

직접 풀어 봐요

1 `7 // 2 ** 2`

```
1. 2 ** 2 = 4
2. 7 // 4 = 1
```

1

2 `10 % 3 ** 2`

```
1. 3 ** 2 = 9
2. 10 % 9 = 1
```

1

3 `2 + 3 * 4 ** 2 // 8`

```
1. 4 ** 2 = 16
2. 3 * 16 = 48
3. 48 // 8 = 6
4. 2 + 6 = 8
```

8

4 `50 - 5 * 2 ** 3 % 7`

```
1. 2 ** 3 = 8
2. 5 * 8 = 40
3. 40 % 7 = 5
4. 50 - 5 = 45
```

45

셀에 print()로 적고 실행해서 답을 맞춰 봐요.

3. 사칙연산

+= 는 지금 값에 더해서 다시 넣어요

[11]

```
score = 0
score += 10
score += 5
score
```

Python

```
15
```



`score += 10`

← score = score + 10을 줄인 모양

같은 모양으로 `-=` `*=` `/=` 도 있어요

3. 사칙연산

글자에도 + 와 * 를 써요

코드	셀에 적은 결과	
`"kim" + "lee"`	`'kimlee'`	이어 붙여요
`"ha" * 3`	`'hahaha'`	3번 반복해요
`"3" + "3"`	`'33'`	따옴표가 있으면 숫자도 글자
`"3" + 3`	`TypeError: can only concatenate str (not "int") to str`	글자와 숫자는 더할 수 없어요

× ~~`"3" + 3`~~ → ✓ `int("3") + 3` → 6

4. 비교와 논리연산

비교하면 답은 True 아니면 False예요

[1]
```python
age = 25
print(age > 20)
print(age < 20)
print(age == 25)
print(age != 25)
print(age >= 25)
print(age <= 24)
```

```
True
False
True
False
True
False
```

기호	뜻	age = 25일 때	결과	
>	크다	age > 20	True	
<	작다	age < 20	False	
==	같다	age == 25	True	← = 두 번
!=	다르다	age != 25	False	
>=	크거나 같다	age >= 25	True	
<=	작거나 같다	age <= 24	False	

결과는 2파트에서 본 불리언(bool)이에요.

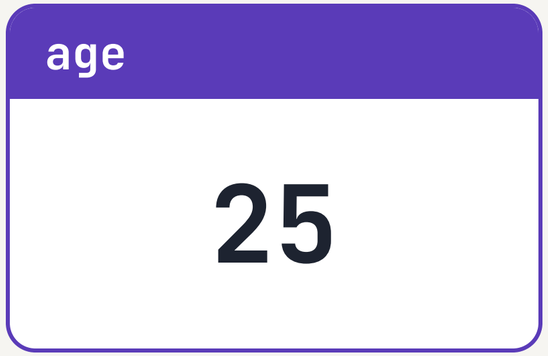
4. 비교와 논리연산

= 하나는 넣기, == 두 개는 묻기

== 묻기

[2]
```
age == 30    ← age가 30이냐고 물어요
```
Python

```
False
```



그대로예요

= 넣기

[3]
```
age = 30    ← 30을 넣어요
age
```
Python

```
30
```



값이 바뀌었어요

비교하려고 = 를 하나만 쓰면 오류 없이 값이 바뀌어요.

4. 비교와 논리연산

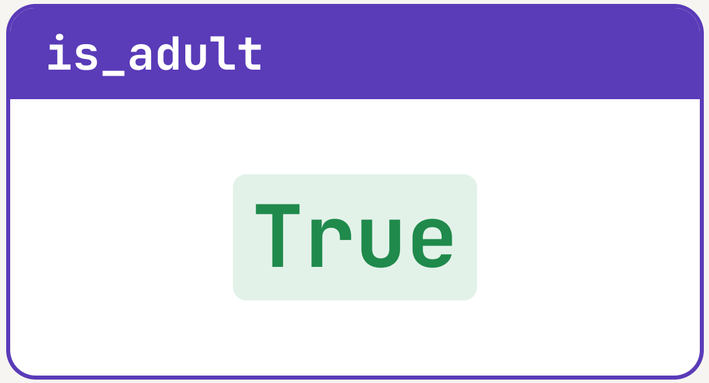
비교한 결과도 변수에 넣을 수 있어요

```
[4]
age = 25
is_adult = age >= 20
print(is_adult)
print(type(is_adult))
```
Python

```
True
<class 'bool'>
```

True, False를 담는 이름은 is_로 시작하면
'~인가요?'로 읽혀요. 10의 is_alive처럼요.





= 오른쪽을 먼저 계산한다는 규칙은 비교에서도 같아요.

and 는 둘 다 True일 때만 True예요

[5]
```python
print(True and True)
print(True and False)
print(False and True)
print(False and False)
```

```
True
False
False
False
```

왼쪽		오른쪽		결과
True	and	True	→	True
True	and	False	→	False
False	and	True	→	False
False	and	False	→	False

True는 이 한 줄뿐이에요

하나라도 False면 결과는 False예요.

or 는 하나만 True여도 True예요

[6]
```python
print(True or True)
print(True or False)
print(False or True)
print(False or False)
```

```
True
True
True
False
```

왼쪽		오른쪽		결과
True	or	True	→	True
True	or	False	→	True
False	or	True	→	True
False	or	False	→	False

False는 이 한 줄뿐이에요

둘 다 False일 때만 False예요.

4. 비교와 논리연산

not 은 True와 False를 뒤집어요

[7]
```
print(not True)
print(not False)
```
Python

```
False
True
```

and, or는 값 두 개 사이에 두고,
not은 값 하나 앞에 붙여요.





not 은 '아니다'예요. 바로 뒤의 값을 반대로 바꿔요.

4. 비교와 논리연산

놀이기구 탑승 조건을 and로 적어요

조건 · 키 120cm 이상 **그리고** 7살 이상

[8]
```python
height = 130
age = 9
height >= 120 and age >= 7
```

```
True
```

130 >= 120 → **True**

9 >= 7 → **True**

True and True → **True**

탈 수 있어요

[9]
```python
height = 110
height >= 120 and age >= 7
```

```
False
```

110 >= 120 → **False**

9 >= 7 → **True**

False and True → **False**

키가 모자라서 못 타요

4. 비교와 논리연산

계산, 비교, not, and, or 순서로 풀어요

```
[10]  2 + 3 > 4 and not False                    Python

      True
```

`2 + 3` > 4 and not False 1 계산

`5 > 4` and not False 2 비교

True and `not False` 3 not

`True and True` 4 and

`True`

헷갈리면 (age >= 20) and (age < 30)처럼 괄호로 묶어요.

먼저 하는 순서

1	`+ - * /`	계산
2	`> == !=`	비교
3	`not`	뒤집기
4	`and`	그리고
5	`or`	또는

5. 카페 주문으로 풀어 보기

가격에 이름을 붙이면 한 줄만 고쳐요

[1]
```python
americano = 4500
latte = 5000
total = americano * 2 + latte
paid = 20000
print("합계:", total)
print("잔돈:", paid - total)
```

← 가격은 맨 위에 한 번만

← 이름으로 읽히는 식

```
합계: 14000
잔돈: 6000
```

← 앞의 숫자 식과 같은 답

[2]
```python
americano = 4800
total = americano * 2 + latte
print("합계:", total)
print("잔돈:", paid - total)
```

← 가격이 오르면 이 줄만

```
합계: 14600
잔돈: 5400
```

← 합계와 잔돈이 같이 바뀌어요

5. 카페 주문으로 풀어 보기

글자로 온 수량을 숫자로 바꿔 나눠 내요

[3]
```python
qty = "3"
total = 4500 * int(qty) + 5000
people = 3
print("합계:", total)
print("한 사람:", total // people)
print("남는 돈:", total % people)
```

← 주문서에서 읽은 수량은 글자예요
← int()로 바꿔서 곱해요 (2파트)

```
합계: 18500
한 사람: 6166
남는 돈: 2
```

← 몫 // 로 한 사람 몫
← 나머지 % 로 남는 돈

[4]
```python
qty = "3"
print(5000 + qty)
```

← int()를 빼면 이렇게 돼요

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

5. 카페 주문으로 풀어 보기

사이즈 업과 할인 조건을 and, or로 적어요

사이즈 업 · 15000원 이상 **그리고** 쿠폰 있음 | 할인 · 회원 **또는** 50000원 이상

[5]

```python
is_member = False
has_coupon = True
total = 18500
size_up = total >= 15000 and has_coupon
discount = is_member or total >= 50000
print("사이즈 업:", size_up)
print("할인:", discount)
print("비회원:", not is_member)
```

Python

← True and True

← False or False

```
사이즈 업: True
할인: False
비회원: True
```

← 둘 다 맞아서 True

← 하나도 안 맞아서 False

← not False · 회원이 아닌가요?

정리

막히면 오류의 마지막 줄을 봐요

마지막 줄	고치기
`NameError: name 'Age' is not defined` 대문자, 소문자를 다르게 적었거나 넣는 셀을 안 돌렸어요	**철자 확인, Run All**
`NameError: name 'true' is not defined` True, False는 첫 글자가 대문자예요	**`True` `False`**
`TypeError: can only concatenate str (not "int") to str` 따옴표 글자와 숫자를 더했어요	**`int("3") + 3`**
`ValueError: invalid literal for int() with base 10: '삼'` int()가 숫자로 못 읽는 글자예요	**소수 글자는 `float()`**
`ZeroDivisionError: division by zero` 0으로 나눴어요	**나누는 수 확인**
비교했는데 값이 바뀌었어요 = 를 하나만 썼어요	**`==` 두 개**

한 장 요약

하고 싶은 것	코드
값에 이름 붙이기	`price = 4500`
값 보기	`print("합계:", total)`
타입 확인	`type(price)`
타입 바꾸기	`int("3")` `str(3)`

하고 싶은 것	코드
계산하기	`+  -  *  /  //  %  **`
먼저 할 계산	`(2 + 3) * 4`
값 쌓기	`score += 10`
비교하기	`==  !=  >  <  >=  <=`

`and` 둘 다 True · `or` 하나라도 True · `not` 뒤집기

푸는 순서는 괄호, 계산, 비교, not, and, or · 헷갈리면 괄호로 묶어요

다음 13에서는 문자열을 자르고, 주문 여러 개를 리스트에 담아요.